

INVESTIGATION REPORT

Issue 333: Reading a debug-only CSR outside debug mode must trap (closed bug, replayed)

openhwgroup/cv32e40x

Document	RTL-TRIAGE-CV32E40X-333
Disposition	Bug confirmed, fix verified
Record date	2026-09-10
Classification	Internal working document. Not a verification sign-off.

1 Status

Issue	openhwgroup/cv32e40x#333
Issue filed	2021-12-01
Issue type	Bug report from an integrator or user
Affects	cv32e40x
Configuration	pre-fix RTL 4ac00cd2 (2021-11-25) vs master; X_EXT=0
Disposition	Bug confirmed, fix verified
Severity	Not assessed
Agreement	Same disposition as the independent human record.
Evidence	6 cited, 2 present, 4 not retained

Disposition meaning. The reported behaviour was reproduced, and the checks listed under Evidence pass against the corrected design.

Severity is not recorded for this run. The harness does not infer a severity from the disposition, so the field is reported as not assessed rather than filled in.

2 Description

Confirmed on the pre-fix RTL and verified fixed on master. Property (controller_fsm scope): a CSR instruction (opcode 0x73, funct3 != 0) addressing dcsr/dpc/dscratch0/dscratch1 (0x7B0-0x7B3) that retires in WB while not in debug mode must raise an illegal-instruction exception, unless the core is entering debug on that instruction (trigger match / DEBUG_TAKEN). Environment: no instruction/data bus errors. On RTL 4ac00cd2 (2021-11-25) BMC finds an 8-cycle counterexample: csrrs x0, dscratch0, x0 (0x7b202073) in M-mode; cs_registers flags csr_illegal_o=1 in EX at cycle 6, yet the instruction retires in WB at cycle 7 with no exception (no debug, no trigger, no interrupt, no bus error). On master the property is proven for all time (abc pdr) and the covers show the same instruction traps (c_hit_traps at cycle 7) while the same access in debug mode is legal (c_dbg_ok). Five property iterations were needed, each diagnosed from a trace: v1 used the pipeline csr_en flag (vacuous: the flag is cleared for illegal ops), v2 used the instruction word, v3 added quiet-bus assumptions (solver had injected an instruction-bus error, cause 1 instead of 2), v4 required wb_valid and excluded ctrl_fsm_ns==DEBUG_TAKEN, v5 also excluded ctrl_fsm_cs==DEBUG_TAKEN and trigger_match_in_wb (solver armed a trigger on the CSR instruction's address, a legal debug entry that suppresses the exception). Comparison with the maintainers' fix, read afterwards (commit a34730a6, 2021-12-01, PR #335 'Fixed issues #332 and #333, where illegal CSR instructions were not properly handled'): in the EX stage the qualifier was csr_illegal_i && (xif_en && !xif_accepted), which with X_EXT=0 is identically false, so the illegal flag never reached WB; the fix inverts it to csr_illegal_i && !(xif_en && xif_accepted). That is exactly the gap the counterexample shows (flag set in EX, no exception in WB).

3 Conditions

Not recorded as separate fields for this run. The conditions and environment assumptions that bound this result are stated inside the Description, and the configuration is given in Status.

4 Implications

Not recorded as a separate field for this run. What the disposition means for a user of the core follows from the Disposition meaning in Status and the bounds in Limitations and scope.

5 Action

Checker used

checker_333.sv

6 Evidence

Ref	Type	Path, relative to the run directory	File	Result recorded
E1	counterexample	formal/prefix_bmc5/job/engine_0/trace.vcd	not kept	pre-fix: csr_illegal_o=1 in EX (cycle 6), instruction retires in WB (cycle 7) without exception; cex_prefix_dbgcsr.png
E2	proof	formal/master_prove5/sby.log	present	master: abc pdr unbounded PASS
E3	proof	formal/master_cover4/sby.log	present	master: covers hit@7, hit_traps@7 (exception taken), dbg_ok@8 (legal in debug mode)
E4	counterexample	formal/master_bmc4/job/engine_0/trace.vcd	not kept	diagnostic (v4): trigger match on the CSR instruction address -> DEBUG_TAKEN suppresses the exception legitimately; led to v5
E5	counterexample	formal/master_bmc3/job/engine_0/trace.vcd	not kept	diagnostic (v3): instruction-bus error injected by the solver -> cause INSTR_FAULT; led to quiet-bus assumption
E6	diff	cores/cv32e40x_hist (git show a34730a6)	not kept	fix: ex_stage csr_is_illegal qualifier inverted (X_EXT=0 case). Read after proving.

Paths are relative to runs/e40x-333. The File column is a check made when this document was generated. It reports whether the cited file exists, not whether its contents were re-validated. 'not kept' means a repository rule deliberately excludes that artefact, usually a waveform dump; rerun the check to regenerate it. 'MISSING' means the file is absent and was expected to be there.

What each class of artefact establishes

- counterexample. A concrete input sequence produced by the engine. Shows the stated condition is reachable; it is not itself a proof.
- proof. Proof or engine run. Scope depends on the mode: prove is unbounded under the stated model, bmc is bounded to its depth, cover establishes reachability only.
- diff. Source or configuration difference.

Artefact(s) E1, E4, E5, E6 are not retained under repository policy. The result recorded against them stands on the engine log cited alongside; the artefact itself must be regenerated by rerunning the check before it can be inspected.

7 Limitations and scope

- The fix is verified against the checks listed under Evidence only. This is not a regression sign-off and does not cover the wider test suite.
- This is an internal investigation record. It requires engineering review before it is used as a disposition. No report generator can supply missing evidence.
- Results apply to the configuration and core revision named in Status. A different configuration needs a separate run.
- Public demonstration runs on open cores are not client engagements and not production verification results.

8

Provenance and measurement

Run directory	runs/e40x-333
Title source	Title and configuration read back from reports/issues/cv32e40x_issue_333.pdf, the previously published report for this issue.
Inputs shown	title, body
Human record	PR #335 / commit a34730a6 (2021-12-01): ex_stage csr_is_illegal = csr_illegal_i && !(xif_en && xif_accepted); previously && (xif_en && !accepted), always false without XIF
Assessed confidence	0.9. This is the investigator's own estimate. It is not a measurement and not a reliability figure.
Iterations	5

Generated by reporting/scripts/issue_document.py from runs/e40x-333/verdict.json. The generator typesets recorded data and checks that cited files exist. It does not run, rerun or validate any check, and it cannot supply missing evidence.